Lecture 14: **Classification**

October 12, 2026

in-class: galaxy morphology with a convolutional neural network

ast384c/inclass/lec13_cnn/galaxy_cnn.ipynb

recap: so far we've trained neural networks by minimizing the loss — *i.e.* the gaussian likelihood or posterior

$$\text{log-likelihood of NN: } \log L(\mathbf{w}) = -\sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} + \text{const}$$

log-posterior of NN:

$$\log p(\mathbf{w} \mid y) = -\sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\sigma_w^2} + \text{const}$$

recap: so far we've trained neural networks by minimizing the loss — *i.e.* the gaussian likelihood or posterior

$$\text{log-likelihood of NN: } \log L(\mathbf{w}) = -\sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} + \text{const}$$

log-posterior of NN:

$$\log p(\mathbf{w} \mid y) = -\sum_i \frac{(y_i - f(x_i; \mathbf{w}))^2}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\sigma_w^2} + \text{const}$$

what if the output we want is a **label**? what's the likelihood of a label?

classification

binomial distribution of k successes in n independent yes/no trials with probability π

$$p(k \mid n, p) = \binom{n}{k} \pi^k (1 - \pi)^{n-k}$$

binomial distribution of k successes in n independent yes/no trials with probability π

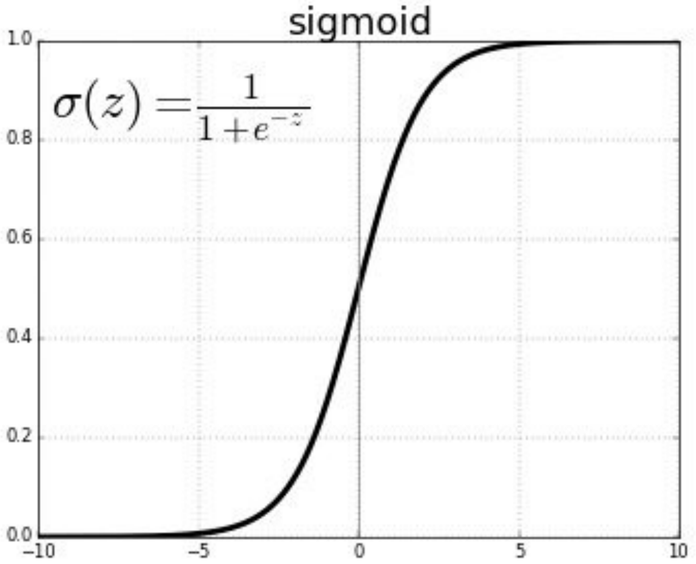
$$p(k \mid n, p) = \binom{n}{k} \pi^k (1 - \pi)^{n-k}$$

$n = 1$ with labels $y \in \{0, 1\}$ — **Bernoulli distribution**

$$p(y \mid \pi) = \pi^y (1 - \pi)^{1-y}$$

neural network with a *sigmoid* activation function in the last year can predict π

$$\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \frac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$$



last lecture

neural network with a *sigmoid* activation function in the last year can predict π

$$\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x}; \mathbf{w})\right) = \frac{1}{1 + e^{-f(\mathbf{x};\mathbf{w})}}$$

$f = \log\left(\frac{\pi}{1-\pi}\right)$ — **logit**, log of how much more likely one outcome is

last lecture

neural network with a *sigmoid* activation function in the last year can predict π

$$\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \frac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$$

for N independent labels, we can write down the likelihood

$$\log L(\mathbf{w}) = \sum_{i=1}^{N}\left[y_i \log \pi_i + (1-y_i)\log(1-\pi_i)\right]$$

neural network with a *sigmoid* activation function in the last year can predict π

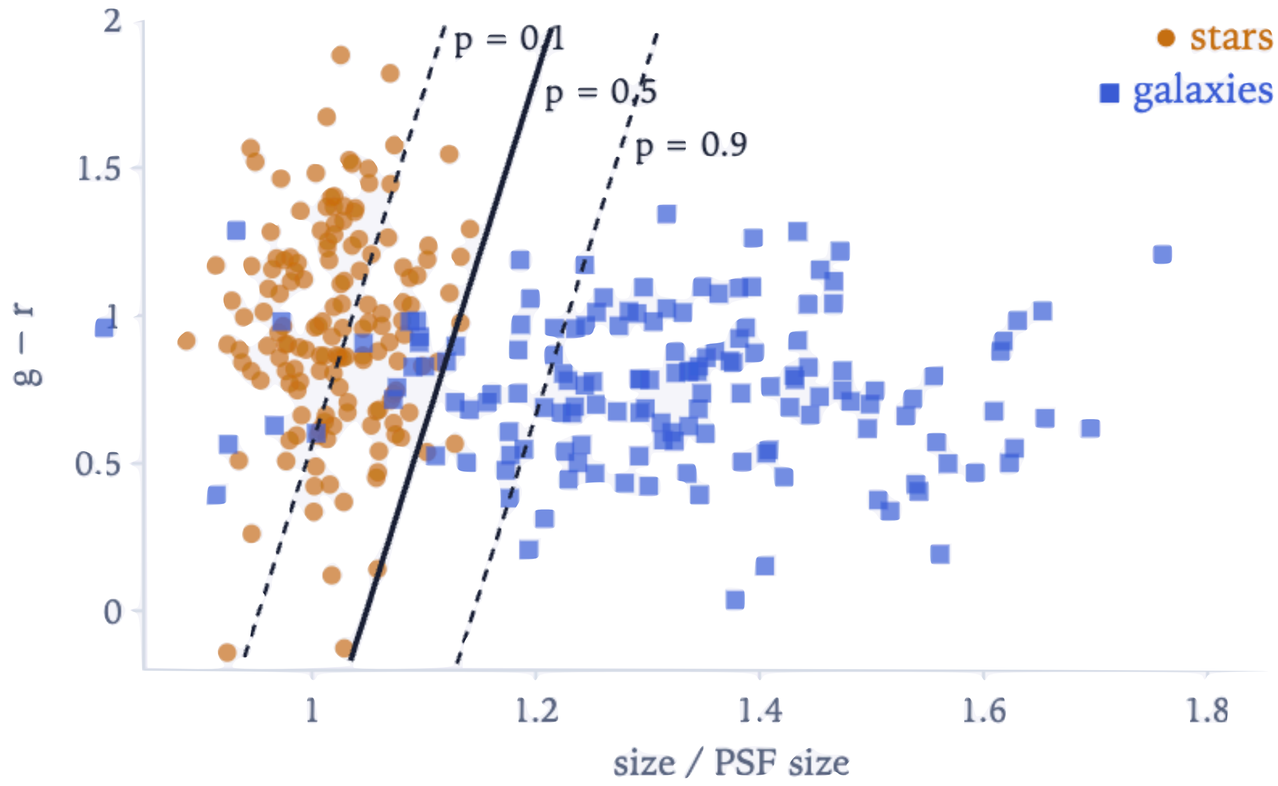
$$\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \frac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$$

for N independent labels, we can write down the likelihood

$$\log L(\mathbf{w}) = \sum_{i=1}^{N}\left[y_i \log \pi_i + (1-y_i)\log(1-\pi_i)\right]$$

binary cross-entropy

simplest classifier: logistic regression — $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}\,\mathbf{x} + b$



constant probability is linear

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and

$\sum_k \pi_k = 1$ — categorical distribution

instead of $\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \dfrac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and $\sum_k \pi_k = 1$ — categorical distribution

instead of $\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \dfrac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$

$$\pi_k(\mathbf{x}_i) = \frac{e^{z_{i,k}}}{\sum_{j=1}^{K} e^{z_{i,j}}} \text{ where } \mathbf{z}_i = f(\mathbf{x}_i;\mathbf{w})$$

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and $\sum_k \pi_k = 1$ — categorical distribution

instead of $\pi(\mathbf{x}) = \sigma\left(f(\mathbf{x};\mathbf{w})\right) = \dfrac{1}{1+e^{-f(\mathbf{x};\mathbf{w})}}$

$$\pi_k(\mathbf{x}_i) = \frac{e^{z_{i,k}}}{\sum_{j=1}^{K} e^{z_{i,j}}} \text{ where } \mathbf{z}_i = f(\mathbf{x}_i;\mathbf{w})$$

$$p(y \mid \pi) = \prod_{i=1}^{k} \pi_k^{y_k}$$

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and $\sum_k \pi_k = 1$ — categorical distribution

$$\log L(\mathbf{w}) = \sum_i^N \log p(y_i; \mathbf{w}) = \sum_i^N \sum_k^K t_{i,k} \log \pi_k(\mathbf{x}_i; \mathbf{w})$$

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and $\sum_k \pi_k = 1$ — categorical distribution

$$\log L(\mathbf{w}) = \sum_i^N \log p(y_i; \mathbf{w}) = \sum_i^N \sum_k^K t_{i,k} \log \pi_k(\mathbf{x}_i; \mathbf{w})$$

$t_{i,k} = 1$ if object i is in label k; otherwise 0 (*one-hot label)*

$t_{i,k}$ can also be fractions (*soft label*)

for labels $y \in \{1,\ldots,K\}$, probability of label k is π_k and $\sum_k \pi_k = 1$ — categorical distribution

$$\log L(\mathbf{w}) = \sum_i^N \log p(y_i; \mathbf{w}) = \sum_i^N \sum_k^K t_{i,k} \log \pi_k(\mathbf{x}_i; \mathbf{w})$$

cross-entropy

name comes from information theory more on this in later lectures

classifier in pytorch

```
model = nn.Sequential(..., nn.Linear(64, 3))
```

```
loss = F.cross_entropy(model(x), y)
```

```
probs = torch.softmax(model(x), dim=1)
```

classifier in pytorch

```
model = nn.Sequential(..., nn.Linear(64, 3))
```

neural network outputs logits for 3 classes

```
loss = F.cross_entropy(model(x), y)
```

```
probs = torch.softmax(model(x), dim=1)
```

classifier in pytorch

```
model = nn.Sequential(..., nn.Linear(64, 3))
```

neural network outputs logits for 3 classes

```
loss = F.cross_entropy(model(x), y)
```

labels: $y = 0,1,2$

```
probs = torch.softmax(model(x), dim=1)
```

classifier in pytorch

```
model = nn.Sequential(..., nn.Linear(64, 3))
```

neural network outputs logits for 3 classes

```
loss = F.cross_entropy(model(x), y)
```

labels: $y = 0,1,2$

```
probs = torch.softmax(model(x), dim=1)
```

apply softmax to convert outputs to probabilities

classifier in pytorch

```
model = nn.Sequential(..., nn.Linear(64, 3))
```

neural network outputs logits for 3 classes

```
loss = F.cross_entropy(model(x), t)
```

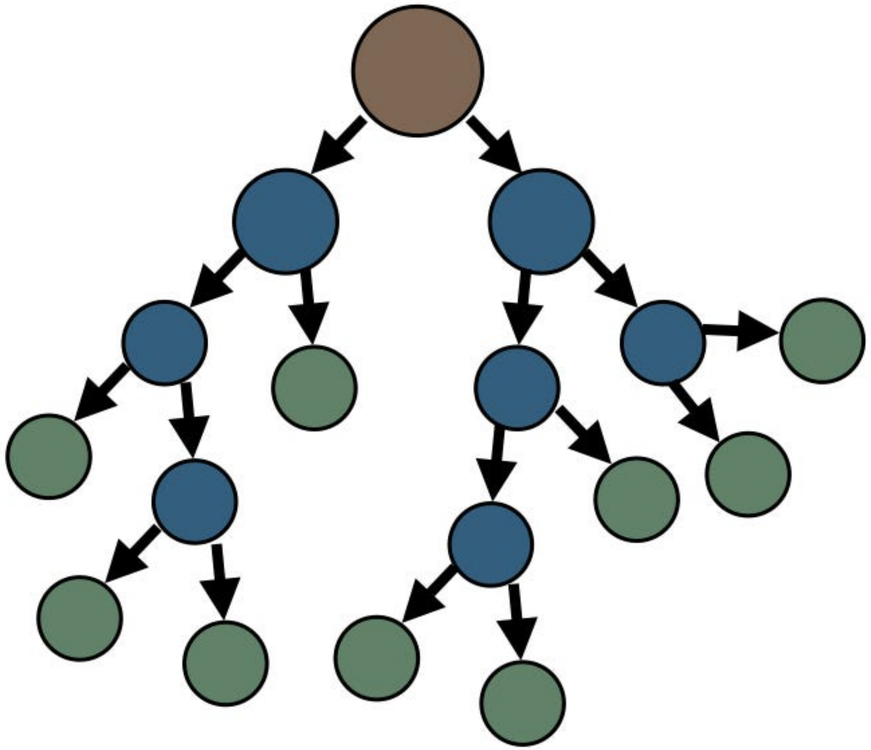
soft labels: t is (N,3) matrix with the fractions

```
probs = torch.softmax(model(x), dim=1)
```

apply softmax to convert outputs to probabilities

some other classifiers

decision trees and forests

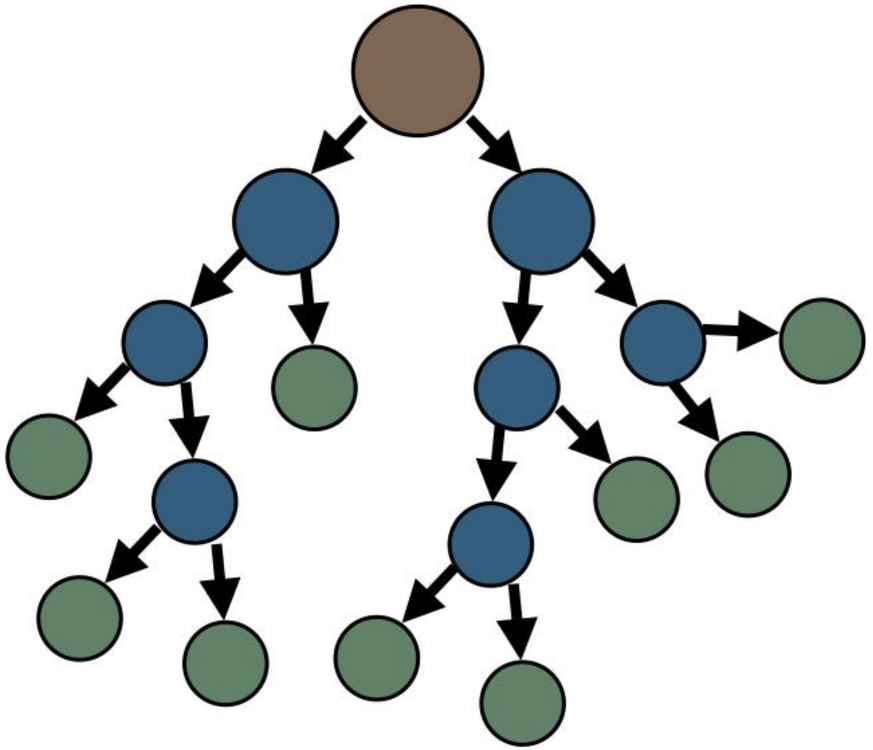


sequence of yes/no question that end in leaves

each leaf predicts the class fraction of objects that end up there

image: Shaw Talebi

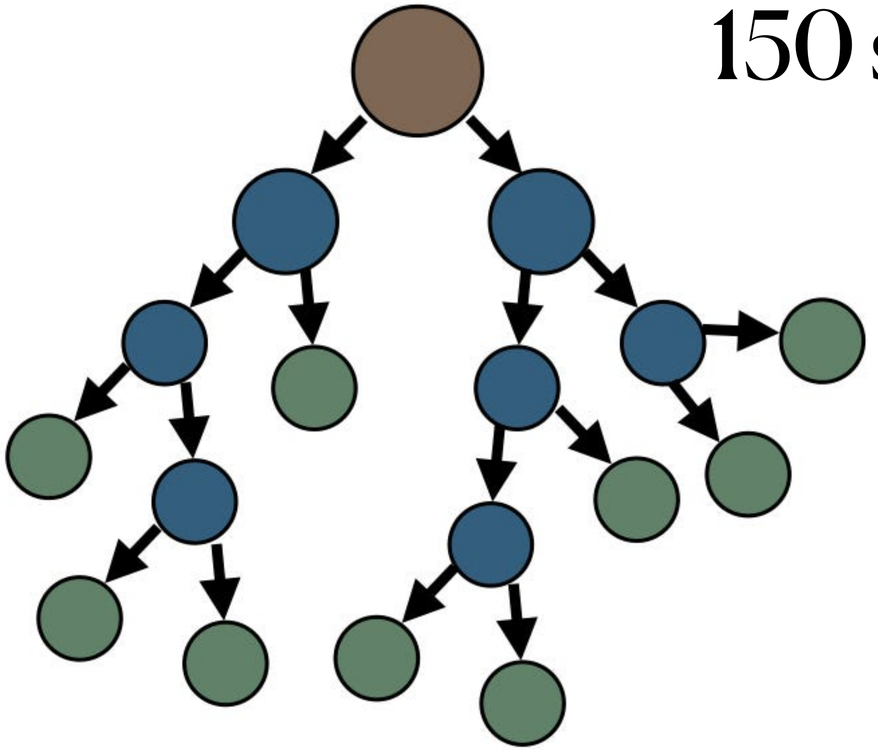
decision trees and forests



$$\text{Gini index} \sum_{k} p_k(1 - p_k)$$

image: Shaw Talebi

decision trees and forests

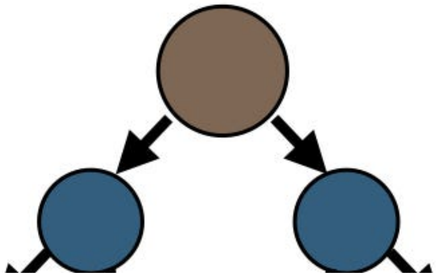


150 stars, 150 galaxies

$$\text{Gini index} \sum_{k} p_k(1 - p_k)$$

image: Shaw Talebi

decision trees and forests



150 stars, 150 galaxies

$G = 1 - 2 \times 0.5^2 = 0.5$

149 stars, 20 galaxies

$G = 0.209$

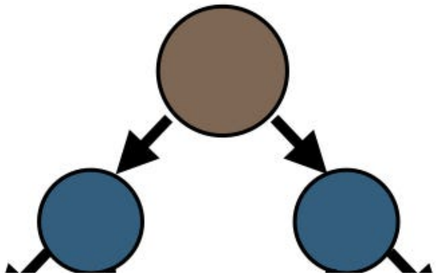
1 stars, 130 galaxies

$G = 0.015$

best split is decided based on lowest weighted Gini index from nodes at the top

image: Shaw Talebi

decision trees and forests



150 stars, 150 galaxies

$G = 1 - 2 \times 0.5^2 = 0.5$

149 stars, 20 galaxies

$G = 0.209$

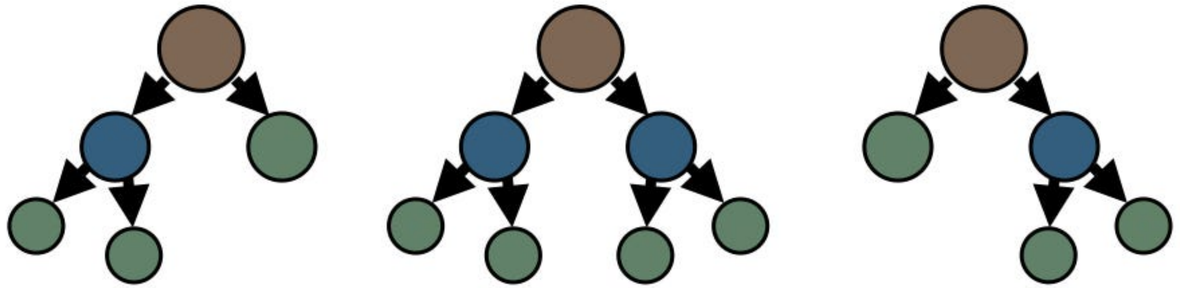
1 stars, 130 galaxies

$G = 0.015$

deep tree can be 100% right on training data but poor performance on test data

image: Shaw Talebi

decision trees and **random forests** (*Breiman 2001*)



1. **boostrap:** draw N objects from N training objects with replacement
2. **random feature:** grow a deep tree on the bootstrap sample but choose splits among m randomly chosen features (*e.g.* $m = \sqrt{d}$)
3. **average** over the trees $p_k(\mathbf{x}) = \frac{1}{B} \sum_b p_k^{(b)}(\mathbf{x})$

image: Shaw Talebi

scoring: **confusion matrix**

		predicted		
		smooth	*edge-on*	*spiral*
true	*smooth*	387	0	17
	edge-on	9	361	12
	spiral	15	13	386

scoring: **confusion matrix**

		predicted *smooth*	*edge-on*	*spiral*	**recall**
	smooth	387	0	17	0.958
true	*edge-on*	9	361	12	
	spiral	15	13	386	

scoring: **confusion matrix**

		predicted *smooth*	*edge-on*	*spiral*	**recall**
true	*smooth*	387	0	17	0.958
	edge-on	9	361	12	
	spiral	15	13	386	
	precision	0.942			

scoring: **confusion matrix**

		predicted *smooth*	*edge-on*	*spiral*	**recall**
	smooth	387	0	17	0.958
true	*edge-on*	9	361	12	**accuracy** 0.945
	spiral	15	13	386	
	precision	0.942			

scoring: **confusion matrix**

		predicted *smooth*	*edge-on*	*spiral*	**recall**
	smooth	387	0	17	0.958
true	*edge-on*	9	361	12	
	spiral	15	13	386	
	precision	0.942			

accuracy 0.945

$$\textbf{F1 score} = \frac{2}{1/\text{recall} + 1/\text{precision}}$$

PSA: don't use accuracy for rare classes — if 1 object in 1,000 is a strong lens, always "not a lens" is 99.9% accurate

purity and completeness of the rare class

image: Shaw Talebi

ROC (Receiver Operating Characteristic) curve

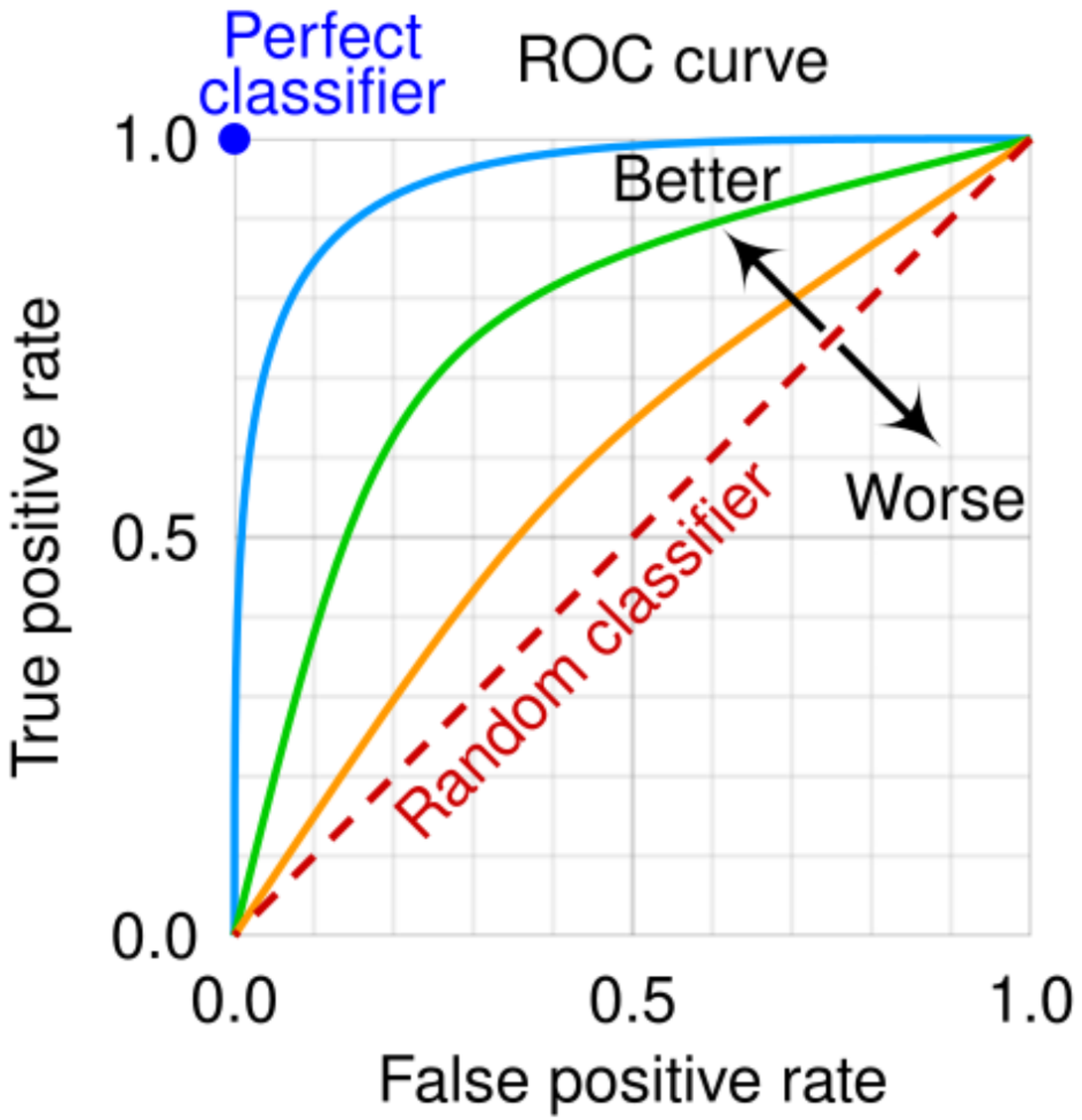


image: wikipedia

in-class: classify galaxies using a convolutional neural network

ast384c/inclass/lec14_classification//galaxy_classifier.ipynb